Introduction to Artificial Intelligence

Final Project

Machine Learning on Auto MPG and CIFAR-10

Yuval Yerushalmi
Student ID: 215838442

March 2026

GitHub Repository:
https://github.com/Yuvalinnnn/intro-to-ai-project

Contents

Introduction **3**

1 Part 1: Regression – Auto MPG **3**
1.1 Dataset Description and Exploratory Data Analysis . . . 3
1.2 Data Splitting and Preprocessing . . . 4
1.3 Linear Regression . . . 4
1.4 Polynomial Regression . . . 6
1.5 KNN Regression . . . 7
1.6 Optimization Behavior: Batch GD vs. mini-batch SGD . . . 8
1.7 Model Comparison and Final Test Evaluation . . . 9
1.8 Model Comparison and Discussion . . . 9

2 Part 2: Classical Classification – CIFAR-10 **10**
2.1 Dataset Description and Preprocessing . . . 10
2.2 Logistic Regression . . . 11
2.3 Linear SVM . . . 12
2.4 KNN Classification . . . 13
2.5 Model Comparison and Final Test Evaluation . . . 15

3 Part 3: Neural Network with PyTorch – CIFAR-10 **16**
3.1 Training Pipeline . . . 16
3.2 Barebones PyTorch (Part II) . . . 16
3.3 Module API (Part III) . . . 17
3.4 Sequential API (Part IV) . . . 17
3.5 Open-Ended Challenge – BetterNet (Part V) . . . 17
3.5.1 Hyperparameter Search . . . 17
3.5.2 BetterNet Architecture . . . 17
3.6 Results . . . 18

4 Bonus: Neural Network Regression with PyTorch – Auto MPG **18**
4.1 Overview . . . 18
4.2 Architecture . . . 19
4.3 Hyperparameter Search . . . 19
4.4 Final Model Training . . . 19
4.5 Final Test Evaluation and Comparison . . . 20

Conclusion **20**

Introduction

This project applies machine learning techniques to two tasks: regression on the Auto MPG dataset (Parts 1 and Bonus) and image classification on CIFAR-10 (Parts 2 and 3). Each part follows a structured methodology: exploratory data analysis, preprocessing, model training with hyperparameter selection on a validation set, and a single final evaluation on the held-out test set.

1 Part 1: Regression – Auto MPG

1.1 Dataset Description and Exploratory Data Analysis

The Auto MPG dataset contains 398 vehicle records with the goal of predicting fuel efficiency (`mpg`, miles per gallon) from 7 numerical features: `cylinders`, `displacement`, `horsepower`, `weight`, `acceleration`, `model_year`, and `origin`.

Missing values: The horsepower column contained 6 missing values (marked as ?). To avoid data leakage, the dataset was first split into train/validation/test sets, and the missing horsepower values were then imputed using the median computed on the training split only. The same median was applied to the validation and test sets.

Key observations from EDA:

- The MPG distribution is right-skewed, with most vehicles in the 15–30 MPG range.
- `weight`, `displacement`, and `horsepower` show strong negative correlations with `mpg` (heavier, larger-engine cars are less fuel-efficient).
- `model_year` shows a positive correlation with `mpg`, reflecting improvements in fuel efficiency over time.

The feature origin was excluded from the scatter plots as it is a categorical variable taking discrete values (1, 2, 3) representing country of origin, making a scatter plot less informative. It is nonetheless included as a feature in all regression models.

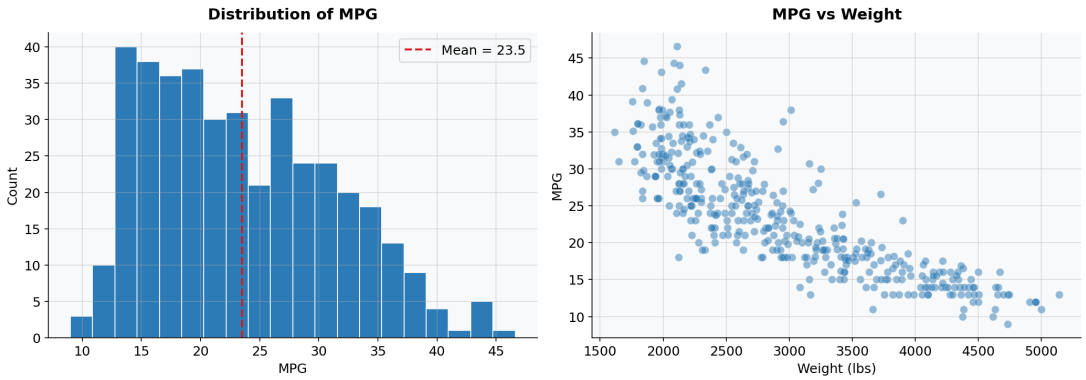

Figure 1: Left: Distribution of MPG. Right: MPG vs. Weight scatter plot.

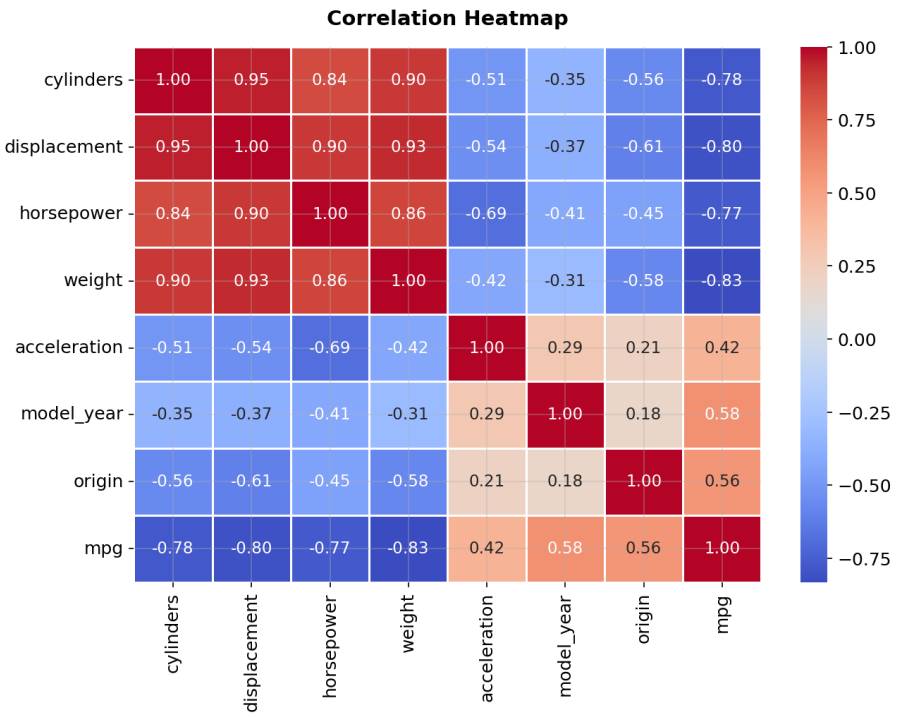

Figure 2: Correlation heatmap of all features and the target variable.

1.2 Data Splitting and Preprocessing

We used the full Auto MPG dataset (398 samples). No samples were removed. The dataset was first split into train/validation/test sets, after which missing values in horsepower were imputed using the median of the training split only. The same value was then applied to the validation and test splits.

The dataset was split into three non-overlapping sets:

- **Train:** 278 samples (70%) - used to fit all models.
- **Validation:** 60 samples (15%) - used exclusively for hyperparameter selection.
- **Test:** 60 samples (15%) - used only once for the final evaluation of the selected model.

All features were standardized using `StandardScaler` fitted on the training set only, and then applied to the validation and test sets, to prevent data leakage.

Standardization is necessary because the features have very different scales (e.g., `weight` vs. `acceleration`), and distance-based models such as KNN are highly sensitive to feature scaling.

1.3 Linear Regression

A standard `LinearRegression` model was trained as a baseline. It assumes a linear relationship between the features and `mpg`.

Validation results:

Metric	Value
RMSE	2.988
MAE	2.360
MSE	8.925
R^2	0.818

Table 1: Linear Regression – Validation Metrics

The model captures roughly 82% of the variance in MPG. The linearity assumption is reasonable as a first approximation, but several feature–target relationships appear only partially linear; in particular, `weight`, `displacement`, and `horsepower` show noticeable non-linear trends. This suggests that a purely linear model cannot fully capture the structure of the data. The residuals also indicate systematic deviations from linearity, especially for heavier and older vehicles.

To better examine the linearity assumption, Figure 3 shows feature–target relationship plots for all numerical features, together with fitted linear trend lines and their equations.

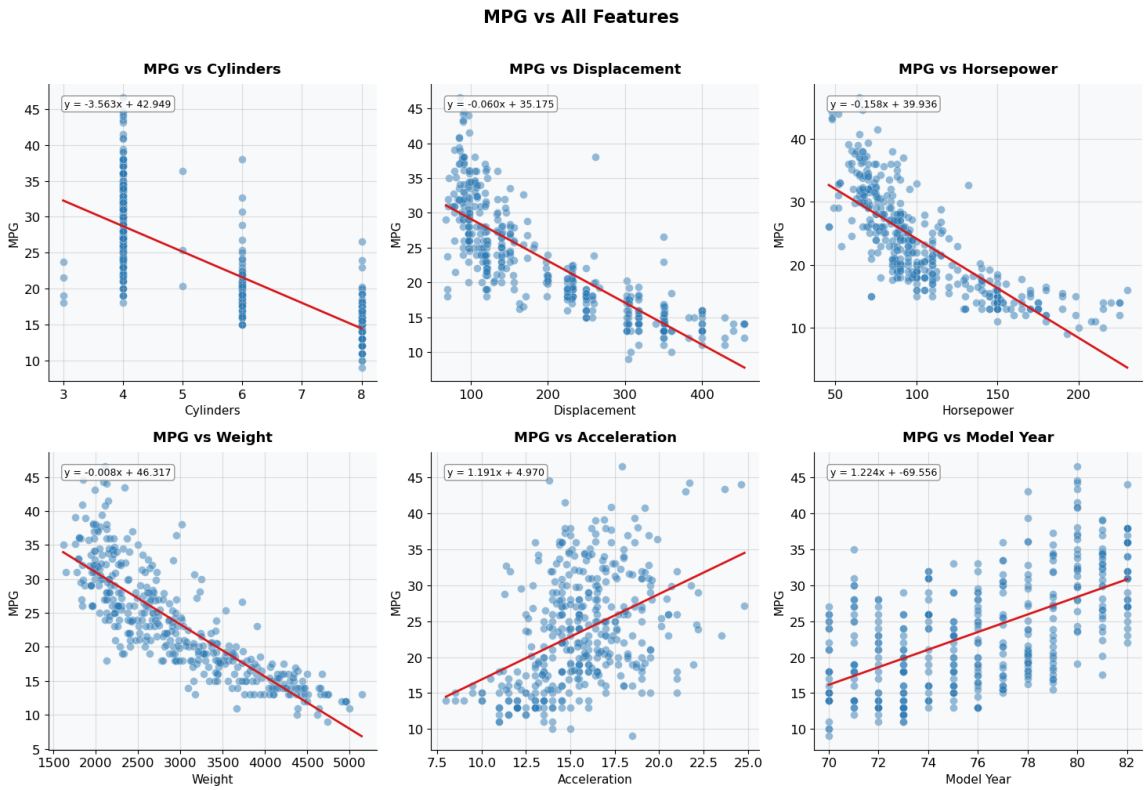

Figure 3: Feature–target relationship plots for the Auto MPG dataset. Each subplot shows MPG versus one feature, together with a fitted linear trend line and its equation.

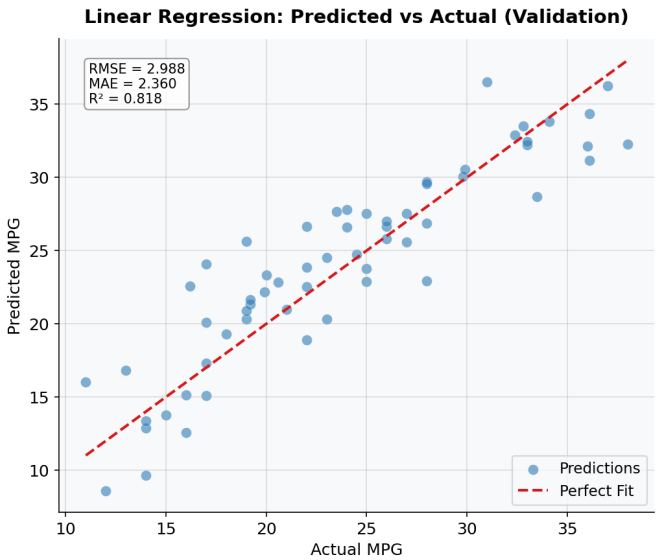

Figure 4: Linear Regression: Predicted vs. Actual MPG on validation set.

1.4 Polynomial Regression

Polynomial features were added to capture non-linear relationships. Degrees 1–5 were evaluated; the degree was selected by minimum validation RMSE.

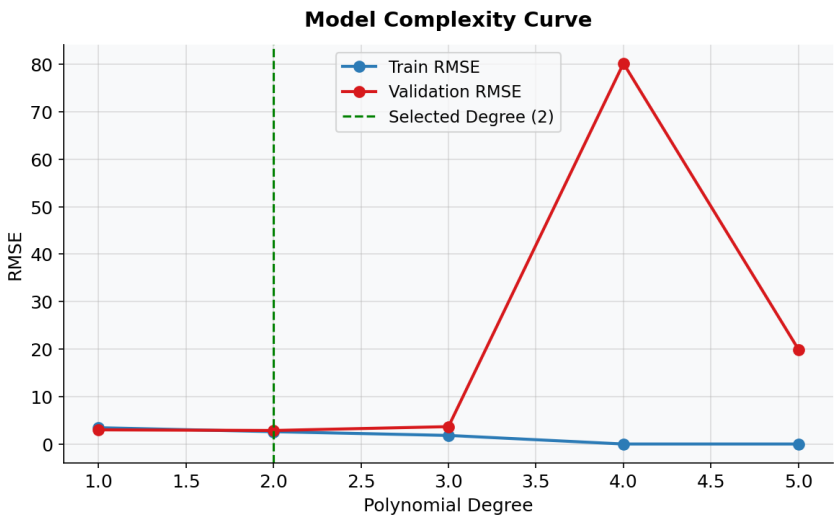

Figure 5: Model complexity curve: Train and Validation RMSE vs. polynomial degree.

Selected degree: 2 (Val RMSE = 2.849). The degree-1 model has relatively high bias, since its limited flexibility prevents it from capturing the non-linear structure in the data; this is reflected in comparatively high training and validation errors. Increasing the polynomial degree reduces bias by making the model more expressive, but it also increases variance. In our results, this effect is evident for degrees 3–5: although the training RMSE becomes very small, the validation RMSE increases sharply, indicating severe overfitting. The quadratic model achieves the best balance between these two effects. It is expressive enough to improve over the linear model, while still maintaining low validation error and good generalization.

This also highlights mild underfitting at degree 1, where the model is too simple to capture the non-linear structure of the data.

1.5 KNN Regression

A `KNeighborsRegressor` was evaluated for $k \in \{1, \ldots, 20\}$.

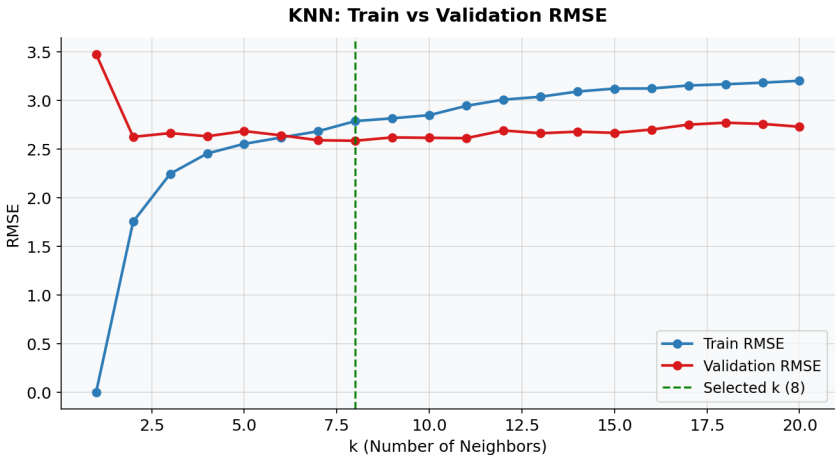

Figure 6: KNN: Train and Validation RMSE vs. number of neighbors k.

Selected k: 8 (Val RMSE = 2.585). At k = 1, the model memorizes the training set (train RMSE ≈ 0) but generalizes poorly. As k increases, variance decreases until k = 8 gives the best validation performance, after which bias starts to dominate.

In KNN regression, the choice of k directly controls the bias–variance trade-off. Very small values of k (especially $k = 1$) lead to low bias but high variance, since the prediction depends heavily on a very small local neighborhood and is therefore sensitive to noise. As k increases, the prediction is averaged over more neighbors, which reduces variance and makes the model more stable. However, if k becomes too large, the model becomes overly smooth, bias increases, and local structure in the data is lost. In our results, $k = 8$ provides the best balance between these two effects, achieving the lowest validation RMSE.

Compared to parametric models, KNN can perform well when the relationship between features and target is strongly non-linear and there is enough local data density to make neighborhood-based predictions reliable. On the other hand, KNN may underperform when the dataset is small, noisy, or high-dimensional, since distances become less informative and predictions become unstable. In the Auto MPG setting, KNN performs well because the dataset exhibits non-linear structure, while the feature dimension is still relatively small.

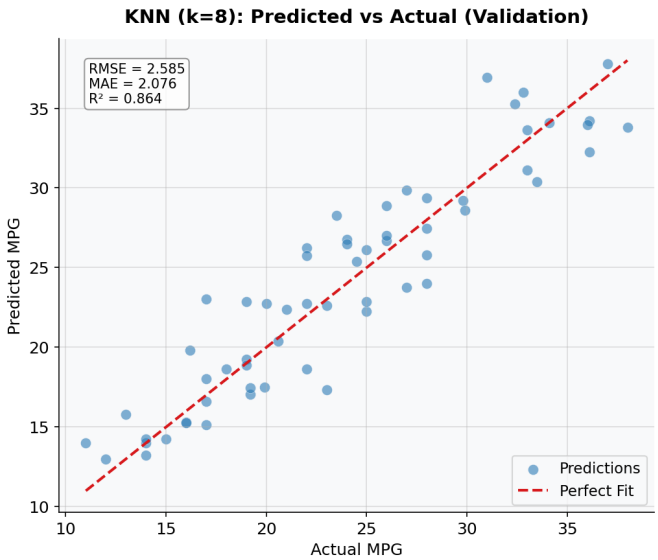

Figure 7: KNN ($k = 8$): Predicted vs. Actual MPG on validation set.

1.6 Optimization Behavior: Batch GD vs. mini-batch SGD

To compare optimization methods, I implemented Batch Gradient Descent and mini-batch SGD on the same linear regression model, and tracked both training and validation RMSE over 200 epochs. In the mini-batch setting, each update was computed using batches of size 16.

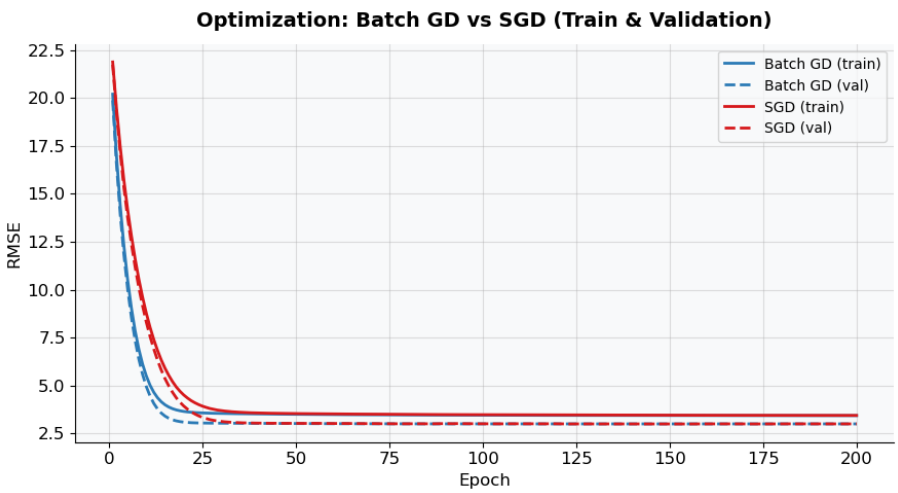

Figure 8: Optimization behavior of Batch Gradient Descent and mini-batch SGD: training and validation RMSE over 200 epochs.

As shown in Figure 8, both methods reduce the RMSE quickly during the first epochs and then converge to a similar final error level. Batch GD produces a smoother and more stable trajectory, since each update is computed using the full training set. In contrast, mini-batch SGD performs more frequent updates based on small batches, so its curve is slightly noisier, especially in the early stages of training.

The validation RMSE is slightly lower than the training RMSE throughout most of the optimization process. This can happen when the validation split is somewhat easier than

the training split, and it does not necessarily indicate a problem. Overall, both methods converge successfully, while Batch GD provides a cleaner and more stable convergence pattern.

1.7 Model Comparison and Final Test Evaluation

Model	**Val RMSE**	**Selected?**
Linear Regression	2.988	
Polynomial (degree=2)	2.849	
KNN ($k = 8$)	**2.585**	✓

Table 2: Model comparison on validation set. KNN achieves the lowest RMSE.

KNN was selected as the final model. After model selection on the validation set, the final KNN model was retrained on the combined train+validation data and evaluated on the test set exactly once:

Metric	**Test Value**
MSE	7.242
RMSE	2.691
MAE	1.957
R^2	0.890

Table 3: KNN ($k = 8$) – Final Test Set Results

The final KNN model achieved test MSE = 7.242, RMSE = 2.691, MAE = 1.957, and $R^2 = 0.890$, confirming that the validation-based model selection generalized well to unseen data.

1.8 Model Comparison and Discussion

Table 2 summarizes the validation RMSE of the three regression models: linear regression, polynomial regression, and KNN regression. In terms of predictive performance, KNN achieved the lowest validation RMSE (2.585), followed by polynomial regression with degree 2 (2.849), while linear regression obtained the highest validation RMSE (2.988). Based on this comparison, KNN was selected as the final model and then retrained on the combined train+validation data befor€e a single test evaluation, where it achieved RMSE = 2.691, MAE = 1.957, and $R^2 = 0.890$.

From a bias–variance perspective, the three models illustrate different trade-offs. Linear regression has the highest bias, since it assumes a purely linear relationship between the features and MPG. This makes it simple and stable, but also limits its ability to capture non-linear patterns in the data. Polynomial regression reduces this bias by increasing model flexibility, but at higher degrees it becomes highly unstable: the training RMSE drops sharply while the validation RMSE increases dramatically, which is a clear sign of high variance and overfitting. KNN regression provides a different type of flexibility:

instead of fitting a global parametric function, it makes local predictions based on nearby samples. With a properly chosen value of k, it balances bias and variance effectively, which explains its strong validation and test performance in this task.

The models also differ in interpretability. Linear regression is the most interpretable, since its coefficients directly reflect the contribution of each feature. Polynomial regression is less interpretable, because the model includes higher-order and interaction terms, making it harder to understand the role of each variable. KNN is the least interpretable from a model-structure perspective, since it does not learn an explicit functional form; instead, predictions are based on local neighborhoods in the training set.

In terms of practical considerations, linear regression is fast to train, easy to explain, and computationally cheap at inference time. Polynomial regression can become numerically unstable and more expensive as the degree increases, especially when unnecessary complexity is introduced. KNN has almost no training cost, but prediction is more expensive because it requires comparing a test point to stored training samples. Still, in this dataset the feature dimension is relatively small, so this limitation is not severe.

If the goal is deployment for this specific task, I would choose the KNN model with $k = 8$. Although it is less interpretable than linear regression, it provides the best overall predictive performance while still remaining practical in this setting. The Auto MPG dataset is relatively small and low-dimensional, which makes KNN a reasonable deployment choice. Therefore, KNN offers the best balance between accuracy and practical usability for this regression problem.

2 Part 2: Classical Classification – CIFAR-10

2.1 Dataset Description and Preprocessing

CIFAR-10 contains 60,000 color images ($32 \times 32 \times 3$ pixels) across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.

Preprocessing pipeline:

1. **Flatten:** Each image is reshaped from $32 \times 32 \times 3$ to a vector of 3,072 features.
2. **Normalize:** Pixel values divided by 255 to bring them into $[0, 1]$.
3. **Standardize:** `StandardScaler` fitted on the training set only, then applied to validation and test sets.

Data splits:

- **Original dataset:** CIFAR-10 contains 50,000 training images and 10,000 test images.
- **Training pool used in this part:** for computational efficiency, only the first 5,000 images from the original training set were used.
- **Train/validation split:** these 5,000 images were split into 4,000 training samples and 1,000 validation samples.
- **Shared model-training subset:** from the 4,000-sample training split, 1,000 samples were randomly drawn with a fixed seed. This same subset was used to train

Logistic Regression, SVM, and KNN during model comparison.

- **Final test set:** the original 10,000-image CIFAR-10 test set was used only once for final evaluation.

Figure 9: One sample image per class from CIFAR-10.

2.2 Logistic Regression

Regularization strength $C \in \{0.001, 0.01, 0.1, 1, 10\}$ was evaluated.

C	Val Accuracy
0.001	**0.366**
0.010	0.294
0.100	0.266
1.000	0.249
10.000	0.247

Table 4: Logistic Regression – Validation Accuracy vs. C

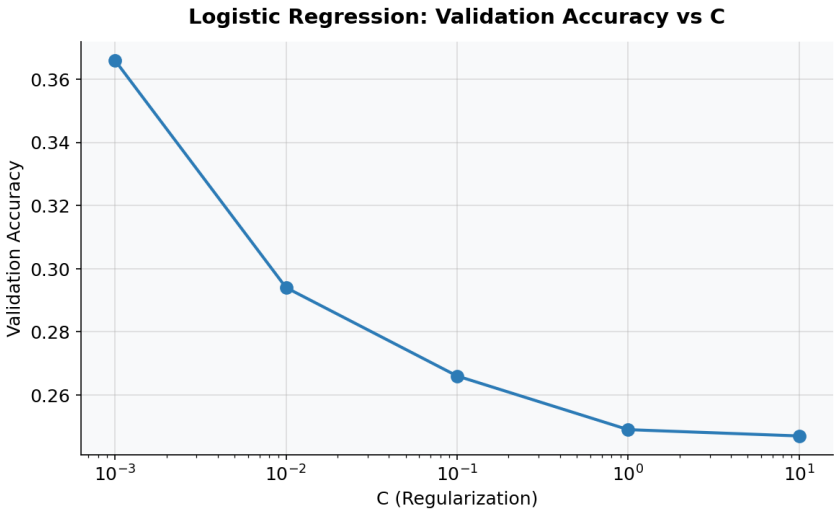

Figure 10: Logistic Regression: Validation Accuracy vs. C

Best $C = 0.001$ (Val Accuracy = 36.6%). Higher values of C reduce regularization, causing overfitting on the small training subset. Strong L2 regularization ($C = 0.001$) generalizes best to the validation set.

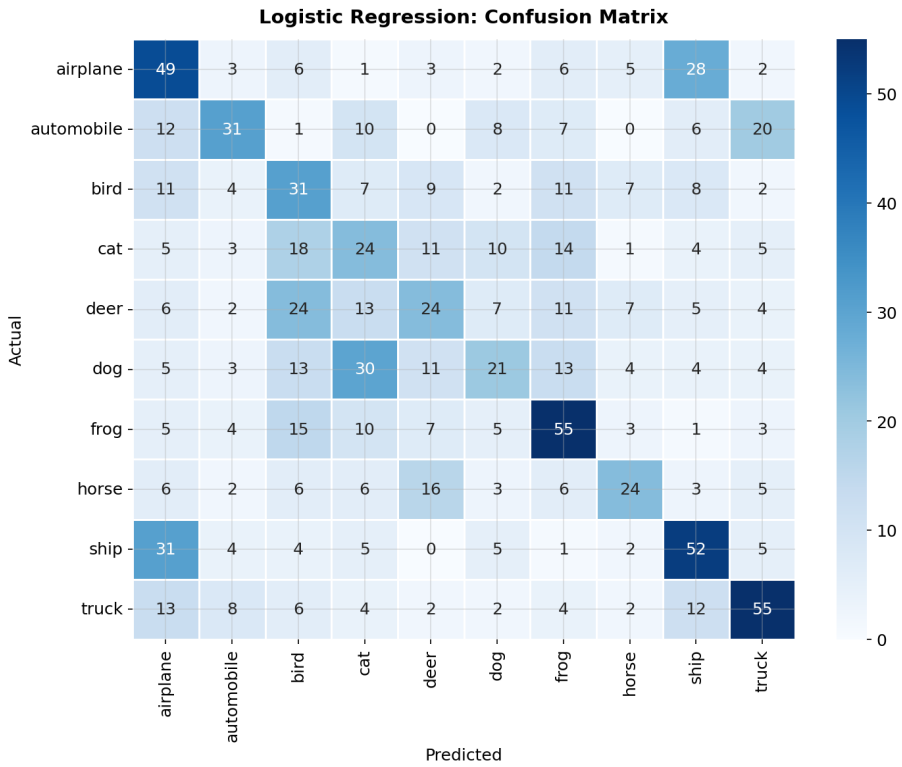

Figure 11: Logistic Regression: Confusion matrix on validation set.

2.3 Linear SVM

Regularization $C \in \{0.001, 0.01, 0.1, 1, 10\}$ was evaluated on 1,000 training samples.

C	Val Accuracy
0.001	**0.282**
0.010	0.235
0.100	0.216
1.000	0.207
10.000	0.210

Table 5: Linear SVM – Validation Accuracy vs. C

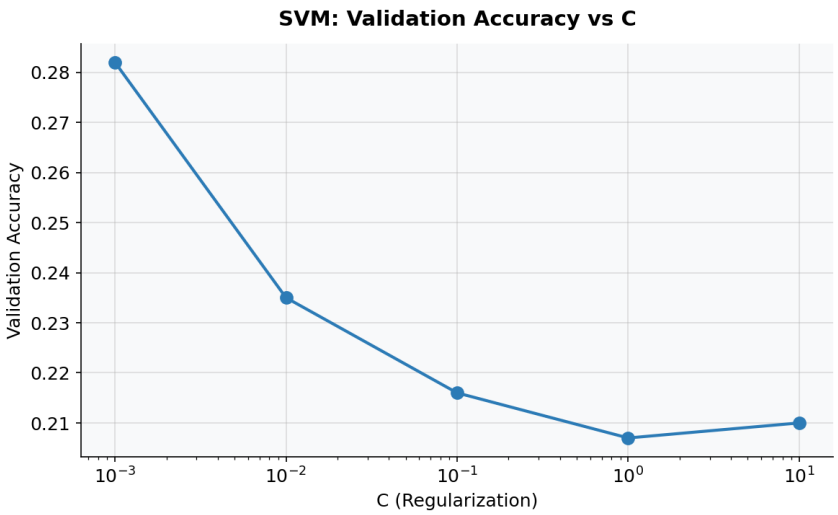

Figure 12: SVM: Validation Accuracy vs. C

Best $C = 0.001$ (Val Accuracy = 28.2%). The same pattern as Logistic Regression: higher regularization (lower C) generalizes better on the small dataset.

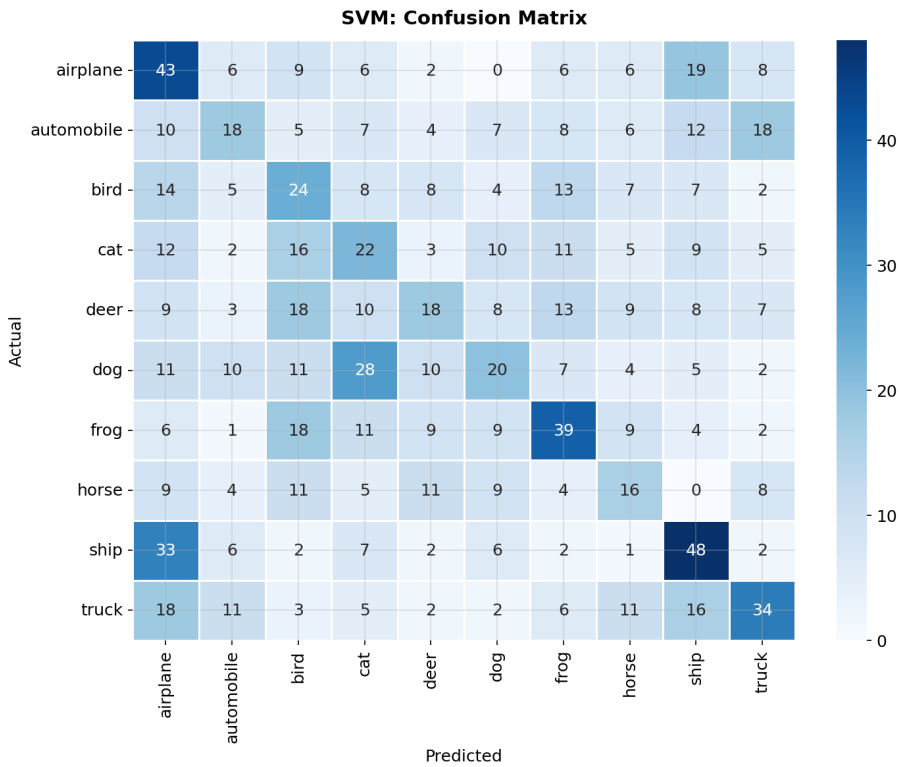

Figure 13: SVM: Confusion matrix on validation set.

2.4 KNN Classification

$k \in \{1, 3, 5, 7, 10\}$ was evaluated on 1,000 training samples.

k	Val Accuracy
1	**0.248**
3	0.229
5	0.240
7	0.236
10	0.210

Table 6: KNN – Validation Accuracy vs. k

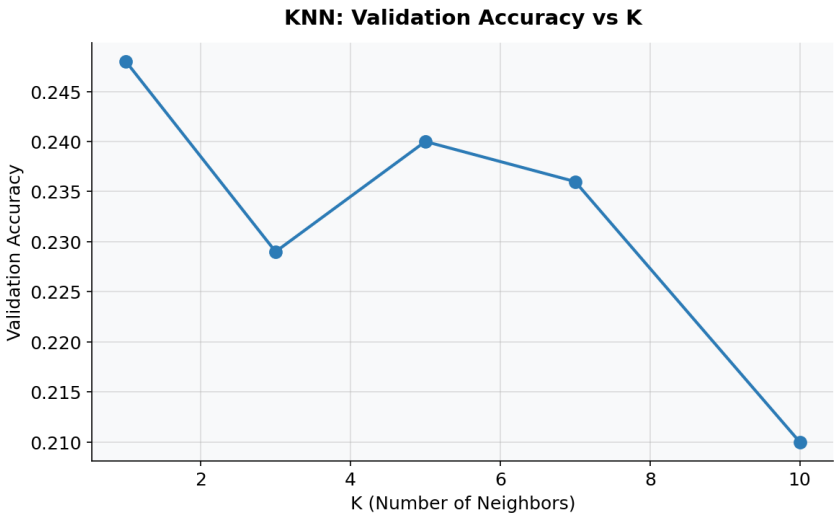

Figure 14: KNN: Validation Accuracy vs. k

Best $k = 1$ (Val Accuracy = 24.8%). With only 1,000 training samples and 3,072 features, the nearest-neighbor distance becomes less discriminative, and all values of k perform poorly.

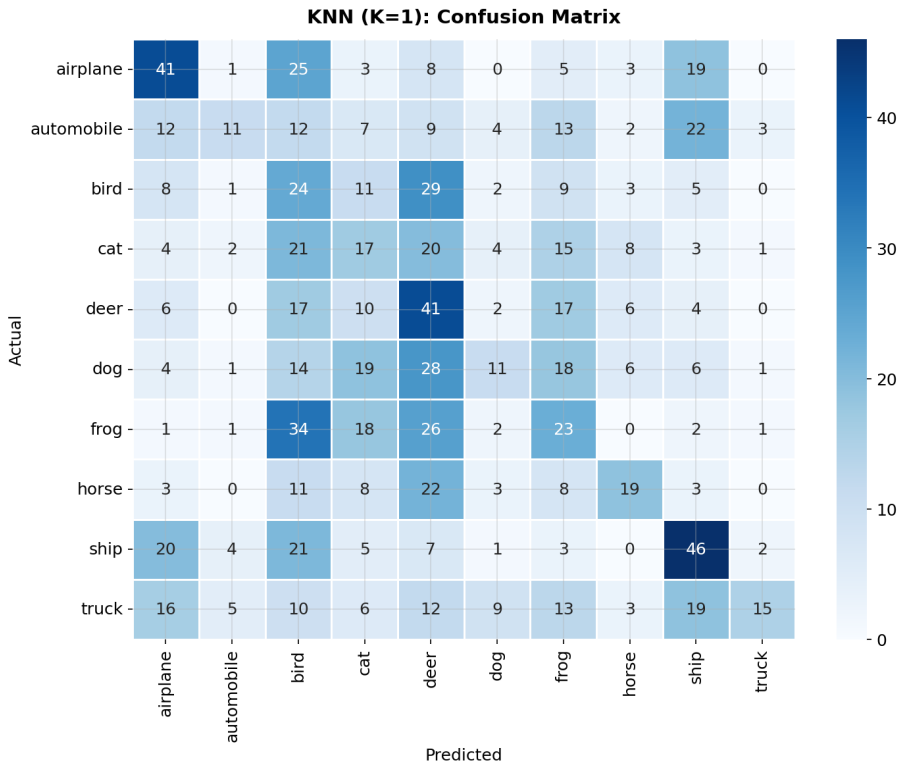

Figure 15: KNN: Confusion matrix on validation set.

2.5 Model Comparison and Final Test Evaluation

Model	Val Accuracy	Selected?
Logistic Regression ($C = 0.001$)	**0.366**	✓
SVM ($C = 0.001$)	0.282	
KNN ($k = 1$)	0.248	

Table 7: Model comparison on validation set.

Logistic Regression achieved the highest validation accuracy and was selected as the final classifier. After model selection, the final Logistic Regression model was retrained on the combined training and validation data (5,000 examples total) and evaluated on the test set exactly once:

Test Accuracy: 37.7%

High input dimensionality: With 3,072 input features (flattened 32×32×3 images), all classical models face significant challenges. KNN is particularly affected: in high-dimensional spaces, distances between points become nearly uniform, making nearest-neighbor search unreliable. Logistic Regression handles this better since it learns a global linear boundary across all features, which explains its superior performance in this setting. Limitations of classical models on image data: A fundamental limitation of all three models is that they treat each pixel as an independent feature, completely ignoring spatial structure. In reality, neighboring pixels are highly correlated and carry important structural information - shapes, edges, textures - that is lost after flattening. This is the primary reason all

models plateau below 40% accuracy. Convolutional neural networks (CNNs), which preserve spatial structure through local receptive fields, would likely achieve 80%+ accuracy on the same dataset.

Logistic Regression outperforms SVM and KNN because it learns a global linear boundary per class across all 3,072 features, while KNN suffers from the curse of dimensionality at this scale.

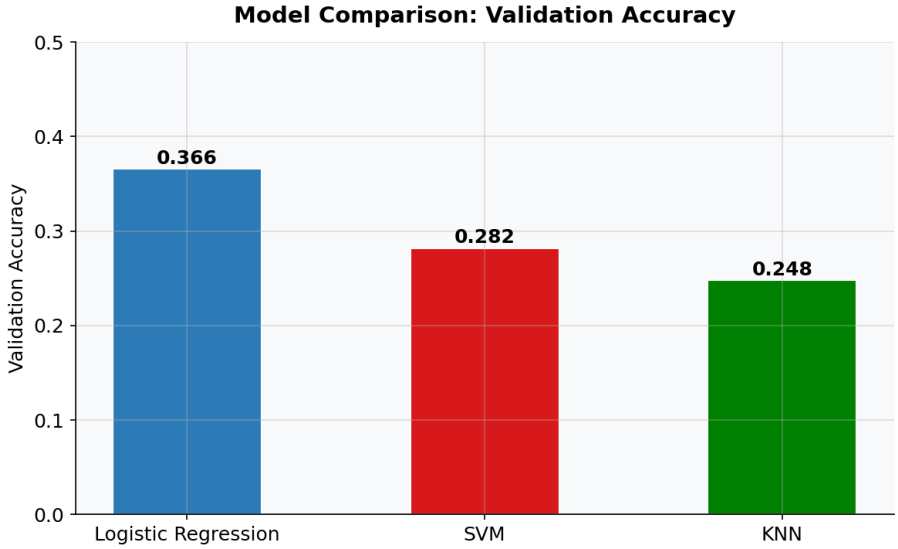

Figure 16: Validation accuracy comparison across all three classifiers.

3 Part 3: Neural Network with PyTorch – CIFAR-10

3.1 Training Pipeline

The CIFAR-10 data was loaded using `torchvision` with standard normalization (mean and std per channel). DataLoaders were created with:

- **Train:** 49,000 samples, batch size 64
- **Validation:** 1,000 samples
- **Test:** 10,000 samples

Loss function: `CrossEntropyLoss`. Accuracy was measured on the validation set at regular intervals during training.

3.2 Barebones PyTorch (Part II)

A two-layer fully-connected network was implemented using raw PyTorch tensors with manual SGD updates (no `nn.Module`). Kaiming initialization was used for weights. Hidden size: 4,000. Validation accuracy after 1 epoch: ∼45%.

3.3 Module API (Part III)

The same network was reimplemented using `nn.Module`, which tracks parameters automatically. `torch.optim.SGD` was used. Validation accuracy after 1 epoch: 38.4%.

3.4 Sequential API (Part IV)

The network was defined with `nn.Sequential` for conciseness, with Nesterov momentum added to the SGD optimizer. Validation accuracy: ∼45%.

3.5 Open-Ended Challenge – BetterNet (Part V)

3.5.1 Hyperparameter Search

To compare several candidate design regimes, we performed a 3-epoch hyperparameter search over 5 configurations, each trained for 3 epochs and evaluated on the validation set:

Config	Hidden	Layers	Dropout	Val Acc (3 epochs)
2-layer small	512	2	0.3	0.524
2-layer medium	1024	2	0.3	0.520
3-layer medium	1024	3	0.3	0.517
4-layer large	2048	4	0.3	0.514
4-layer low dropout	2048	4	0.2	**0.549**

Table 8: Hyperparameter Search – Validation Accuracy after 3 epochs

The 3-epoch hyperparameter search was used as a coarse architecture search over depth, hidden width regime, and dropout. In this search, the best-performing family was a deeper low-dropout network (large hidden size, 4 hidden blocks, dropout = 0.2). Based on this result, I kept the same design regime and performed a targeted architectural refinement to a tapered MLP, $3072 \to 2048 \to 1024 \to 512 \to 10$, for the final 10-epoch model. The refinement was motivated by parameter efficiency and gradual dimensionality reduction, while preserving the main conclusion of the search: deeper architecture with low dropout performs best.

The search results should be interpreted as identifying the best design regime, rather than proving that one exact layer-width schedule is uniquely optimal. The comparison shows that a deeper network with low dropout is preferable to the shallower or more strongly regularized alternatives. Within this regime, reducing dropout from 0.3 to 0.2 improved validation accuracy for the deeper large-width model (0.549 vs. 0.514), suggesting that BatchNorm already provided substantial regularization. The final BetterNet therefore keeps the same overall regime and refines the width schedule into a tapered architecture.

3.5.2 BetterNet Architecture

The final BetterNet preserves the deeper low-dropout design regime identified above, while refining the hidden-layer widths into a tapered architecture.

Layer	Size	Operations
FC 1	3072 → 2048	BatchNorm, ReLU, Dropout(0.2)
FC 2	2048 → 1024	BatchNorm, ReLU, Dropout(0.2)
FC 3	1024 → 512	BatchNorm, ReLU, Dropout(0.2)
FC 4	512 → 10	Output

Table 9: BetterNet Architecture

Optimizer: Adam ($lr = 10^{-3}$, weight_decay = 10^{-4}), 10 epochs. **BatchNorm** normalizes layer inputs, enabling faster convergence and higher learning rates. **Dropout(0.2)** reduces co-adaptation of neurons during training.

3.6 Results

Model	Val Accuracy	Test Accuracy
Barebones (Part II)	∼45%	–
Module API (Part III)	∼38.4%	–
Sequential (Part IV)	∼45%	–
BetterNet (Part V)	∼58%	**55.52%**

Table 10: PyTorch Models – Accuracy Comparison

BetterNet achieved 55.52% test accuracy, clearly outperforming the classical models used in Part 2 within this project setup. However, this comparison is not architecture-only: the neural network was trained on 49,000 images, while the classical models were intentionally restricted to a much smaller 1,000-example training subset for computational efficiency. Thus, the improvement reflects both representation learning and a substantially larger amount of training data.

4 Bonus: Neural Network Regression with PyTorch – Auto MPG

4.1 Overview

We adapt the PyTorch classification pipeline to regression on the Auto MPG dataset. Key differences from Part 3: (1) output is a single scalar; (2) loss function is MSE, chosen because it directly penalizes the squared difference between predicted and actual MPG values, making it the natural choice for regression tasks where the target is a continuous variable; (3) evaluation uses RMSE and R^2 instead of accuracy; (4) both features and targets are standardized with StandardScaler. As in Part 1, missing values in horsepower were imputed using the median of the training split only, and the same value was applied to validation and test.

Data split: 278 train / 60 validation / 60 test (same split as Part 1).

4.2 Architecture

A flexible **RegressionNet** with configurable hidden size, number of layers, and dropout rate. Each hidden layer uses BatchNorm → ReLU → Dropout. Kaiming initialization is applied to all linear layers.

4.3 Hyperparameter Search

Five configurations were evaluated for 50 epochs, selecting the best by validation RMSE:

Hidden	Layers	Dropout	LR	Val RMSE
64	2	0.2	0.001	**2.399**
128	2	0.2	0.001	2.482
128	3	0.3	0.001	2.555
256	3	0.3	0.001	2.388
128	2	0.2	0.0005	2.497

Table 11: Hyperparameter Search – Validation RMSE (50 epochs each)

The best configuration uses hidden size 256, 3 layers, dropout 0.3, and $lr = 10^{-3}$. In this run, the larger network slightly outperformed the smaller alternatives, suggesting that the combination of BatchNorm, Dropout, and Adam was sufficient to control overfitting even on the small training split (278 samples).

4.4 Final Model Training

The best config was trained for 100 epochs with Adam ($lr = 10^{-3}$, weight_decay $= 10^{-4}$). Architecture: FC(7 → 256) → BN → ReLU → Dropout(0.3) → FC(256 → 256) → BN → ReLU → Dropout(0.3) → FC(256 → 256)→ BN → ReLU → Dropout(0.3) → FC(256 → 1).

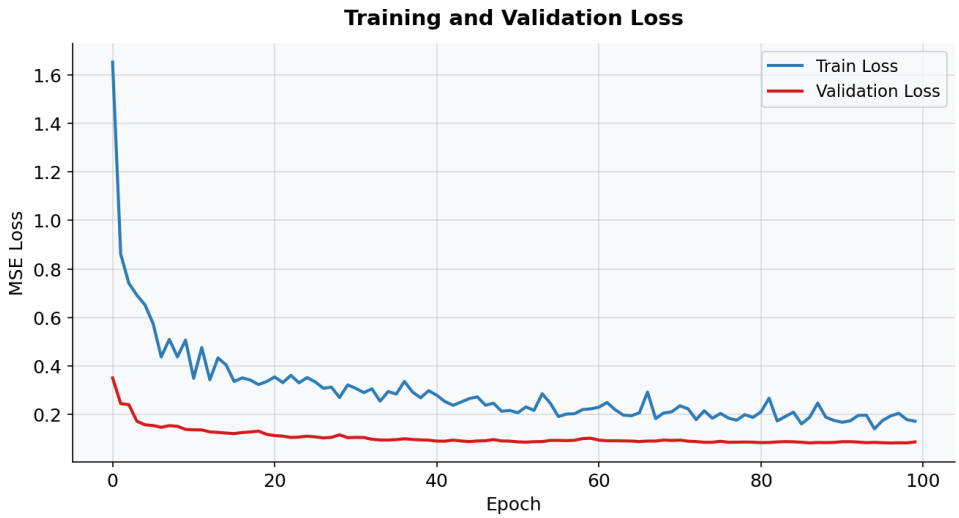

Figure 17: Training and Validation MSE loss over 100 epochs.

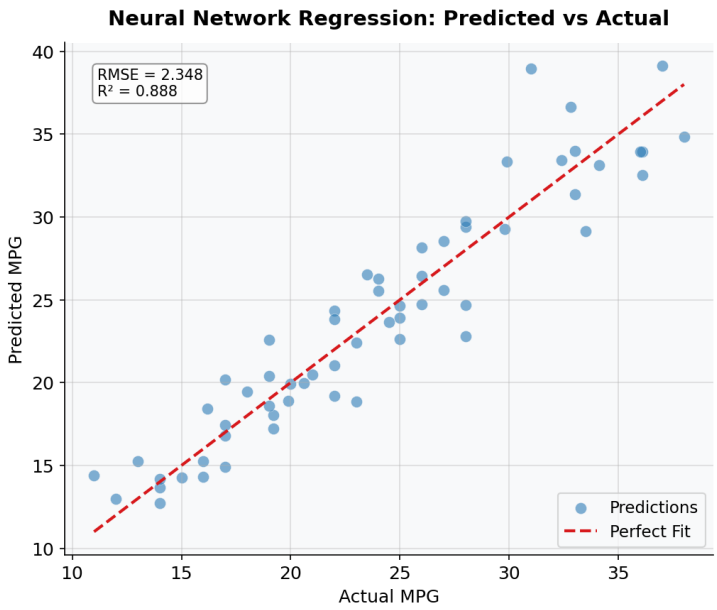

Figure 18: Predicted vs. Actual MPG on validation set.

Validation results: RMSE = 2.383, $R^2 = 0.884$.

4.5 Final Test Evaluation and Comparison

Model	Test RMSE	Test R^2
Linear Regression (Part 1)	–	–
Polynomial deg=2 (Part 1)	–	–
KNN $k = 8$ (Part 1)	2.691	0.890
RegressionNet (Bonus)	**2.383**	**0.914**

Table 12: Regression Model Comparison – Test Set Results

The neural network achieves Test RMSE = 2.383 and $R^2 = 0.914$, outperforming the best classical model from Part 1 (KNN, RMSE = 2.691, $R^2 = 0.890$). The improvement is modest but consistent, suggesting that the neural network captures additional non-linear feature interactions beyond those exploited by KNN.

Conclusion

Part	Best Model	Test Metric	Value
1 – Regression	KNN ($k = 8$)	RMSE	2.691
2 – Classification	Logistic Regression	Accuracy	37.7%
3 – Neural Network	BetterNet	Accuracy	55.52%
Bonus – NN Reg.	RegressionNet (h=256)	RMSE	2.383

Table 13: Summary of best models and test results across all parts.

Key insights:

Regression (Parts 1 & Bonus): KNN outperformed linear and polynomial regression because Auto MPG contains non-linear feature interactions (e.g., the relationship between weight and mpg is not linear). The neural network further improved performance (RMSE 2.383 vs. 2.691) by learning more complex non-linear mappings, despite the small dataset.

Classical Classification (Part 2): Logistic Regression outperformed SVM and KNN on CIFAR-10. At 37.7% test accuracy, all classical models face a fundamental limitation: raw pixel features (3,072 dimensions) contain little spatial structure after flattening, making linear boundaries and nearest-neighbor distances poor discriminators.

Neural Network (Part 3): BetterNet achieved 55.52% test accuracy, outperforming the classical models under the experimental setup used in this project. This comparison should be interpreted with caution, since BetterNet was trained on 49,000 images whereas the classical models in Part 2 were trained on a 1,000-example subset. BatchNorm stabilized training, and Dropout(0.2) helped regularization. The primary limitation is that fully-connected networks on raw pixels still ignore spatial structure; convolutional networks (CNNs) would likely achieve 80%+ on this dataset.

Future improvements: Replacing fully-connected layers with convolutional layers (Part 3), using data augmentation to expand the training set (Part 2), and applying ensemble methods (Parts 1–2) could all yield meaningful performance gains.